

Serving Options Table — Multi-Tone Communication Assistant

Role: Serving

Team Member: Rithwik Amajala (sa9880)

Deadline: April 6, 2026

Overview

The serving stack consists of two microservices: 1. **Classifier** — DistilBERT 3-class tone classifier (Formal / Friendly / Neutral), CPU-based 2. **Generator** — Instruction-tuned LLM (Flan-T5), GPU-based

Latency target: **p95 < 800 ms end-to-end** (Classifier < 100 ms + Generator < 600 ms + API overhead < 100 ms)

Peak load: **~17 requests/second** (200 users × 5 msg/min)

Classifier Serving Options

All experiments run on Chameleon Cloud KVM@TACC (serve-proj15, m1.xlarge, floating IP 129.114.25.168) inside Docker containers.

Benchmark: 200 requests per concurrency level, measured with `benchmark.py`.

Peak load target: concurrency=17 (~17 req/s), p95 < 100 ms.

Option	Endpoint URL	Model	Hardware	p50 (ms)	p95 (ms)	p99 (ms)	Throughput @ c=17 (req/s)	Notes
classifier	POST <code>http://129.114.25.168:8001/predict</code>	distilbert-base-uncased	CPU (m1.xlarge)	63.2	83.9	87.6	125.21	Simplest reference; meets p95 < 100ms target
classifier	POST <code>http://129.114.25.168:8002/predict</code>	distilbert-base-uncased (ONNX Runtime)	CPU (m1.xlarge)	37.9	48.6	52.0	145.24	Best latency; 1.7× faster than baseline at peak load

Option	Endpoint URL	Model	Hardware	p50 (ms)	p95 (ms)	p99 (ms)	Throughput @ c=17 (req/s)	Notes
classifirest	POST	QuantizedDistilBertBase	CPU	46.2	58.5	64.9	102.53	Best re-source effi-ciency; lower RAM foot-print than base-line
□	http://129.168.180.3/predict	uncased	m1.xlarge)					
		INT8						

□ = most promising options. All three meet the p95 < 100 ms classifier target at peak load (concurrency=17).

Recommended for deployment: classifier_onnx — lowest p95 latency (48.6 ms) and highest throughput (145.24 req/s) at peak load.

Generator Serving Options

Dummy mode benchmarked on Chameleon KVM@TACC CPU instance to establish concurrency limits.

Real GPU results (NVIDIA RTX6000 / H100) to be added once GPU lease is active.

Option	Endpoint URL	Model	Hardware	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Notes
generatorrest	POST	dummy_cpu	CPU	605.3	669.9	681.2	1.48	Only mea-sured op-tion; c=1 border-line meets <800ms e2e bud-get
□	http://129.168.180.3/generate	rewriter	m1.xlarge)					
(base-line)								

Option	Endpoint URL	Model	Hardware	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (req/s)	Notes
generator	POST my_cpu_template http://129.115.25.16:8000/generate	rewriter	CPU (KVM m1.xlarge)	1141.9	1704.9	1802.3	1.67	c=2 exceeds latency budget — GPU is mandatory for scale
generator	POST an_t5_gpu_google/16 http://129.115.25.16:8000/generate	base — fp16	GPU (RTX6000) — pending	~320	~490	~560	~2.1	Target deployment; GPU required to meet <600ms generator budget
generator	POST an_t5_gpu_google/16 http://129.115.25.16:8000/generate	base — fp16	GPU (RTX6000) — pending	~380	~540	~610	~3.8	System-level opt: dynamic batching; best throughput on GPU

□ = most promising measured option. GPU rows are estimates pending GPU lease activation.

Key finding from CPU benchmark: Generator requires GPU. At concurrency=2, p95 jumps to 1704.9 ms — well above the 600 ms budget. This empirically justifies the GPU infrastructure requirement.

Right-Sizing Summary

Resource usage measured on Chameleon KVM@TACC m1.xlarge via `docker stats` (idle + under benchmark load).

Service	Instance Type	CPU % (idle)	RAM Usage (idle)	RAM Limit	GPU	Basis
classifier-pytorch	KVM m1.xlarge	0.10%	49.3 MiB	512 MiB	None	Handles 115 req/s at p95=82ms
classifier-kvm	KVM m1.xlarge	0.10%	49.4 MiB	512 MiB	None	Handles 144 req/s at p95=48ms
classifier-quantized	KVM m1.xlarge	0.08%	49.4 MiB	256 MiB	None	recom- mended Runs within 256 MiB limit; most memory-efficient
generator (dummy)	KVM m1.xlarge	0.08%	57.7 MiB	512 MiB	None	Dummy mode only; real LLM requires GPU + 8 GB RAM
generator (real fp16)	GPU (RTX6000)	~15%	~4 GB	8 GB	1× RTX6000	Required for <600ms latency at any concurrency

Optimization Summary

Optimization Type	Technique	Applied To	Measured Effect
Model-level	ONNX export + graph fusion (ORT_ENABLE_ALL)	Classifier	p95: 83.9 ms → 48.6 ms (1.7× faster at peak load c=17)
Model-level	INT8 dynamic quantization	Classifier	p95: 83.9 ms → 58.5 ms (1.4× faster); lower RAM usage
System-level	Dynamic request batching	Generator	~1.8× throughput increase on GPU (pending GPU measurement)
Infrastructure-level	GPU (fp16)	Generator	CPU p95 649ms at c=1 → GPU ~490ms; enables sustainable concurrency
Infrastructure-level	Multi-worker uvicorn	Classifier	Additional throughput headroom for burst traffic

How to Reproduce

```
# 1. Build images
cd serving/
docker compose build classifier-pytorch

# 2. Run baseline classifier
docker compose up classifier-pytorch

# 3. Run benchmark
cd evaluate/
pip install -r requirements.txt
python benchmark.py \
  --url http://<CHAMELEON_IP>:8001/predict \
  --endpoint classifier \
  --concurrency 1 5 10 17 \
  --n 200 \
  --label classifier_pytorch_baseline \
  --out results/summary.csv

# 4. Run ONNX option
docker compose up classifier-onnx # port 8002
python benchmark.py \
  --url http://<CHAMELEON_IP>:8002/predict \
  --label classifier_onnx --out results/summary.csv ...

# 5. Run all at once (after Chameleon IPs are set)
cd evaluate/
./run_all_benchmarks.sh <CLASSIFIER_IP> <GENERATOR_IP>
```